

1. Intro

```
JavaScript is a cross-platform, object-oriented scripting language used to make webpages interactive (e.g., having complex anim
```

- JavaScript borrows most of its syntax from Java, C, and C++, but it has also been influenced by Awk, Perl, and Python.
- It is primarily used for client-side web development, but it is also used on the server-side (Node.js).
- It is an interpreted language, meaning it does not require compilation before execution.

History ✨

JavaScript was created by `Brendan Eich` in `1995` for Netscape Navigator. It has evolved into one of the core technologies of the web.

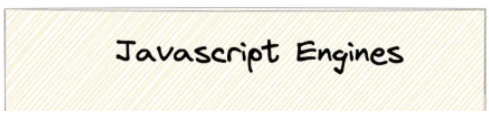
`Evolution`

Year	Version	Key Features	Owner
1995	JS 1.0	Initial release	Netscape
1996	JS 1.1	Added regular expressions	Netscape
1997	ECMAScript 1	Standardization begins	ECMA
1998	ECMAScript 2	Minor updates	ECMA
1999	ECMAScript 3	Added try/catch, regex improvements	ECMA
2009	ECMAScript 5	Strict mode, JSON support	ECMA
2015	ECMAScript 6 (ES6)	Classes, modules, arrow functions	ECMA
2016	ECMAScript 7	Exponentiation operator, Array.prototype.includes	ECMA
2017	ECMAScript 8	Async/await, Object.values/entries	ECMA
2018	ECMAScript 9	Rest/spread properties, Promise.finally	ECMA
2019	ECMAScript 10	Array.flat/flatMap, Object.fromEntries	ECMA
2020	ECMAScript 11	BigInt, Dynamic import, Nullish coalescing	ECMA
2021	ECMAScript 12	Logical assignment operators, String.replaceAll	ECMA
2022	ECMAScript 13	Class fields, Top-level await	ECMA
2023	ECMAScript 14	Pattern matching, WeakRefs	ECMA
2024	ECMAScript 15	Temporal API, Record & Tuple	ECMA
2025	ECMAScript 16	Pipeline operator, Pattern matching enhancements	ECMA

Features

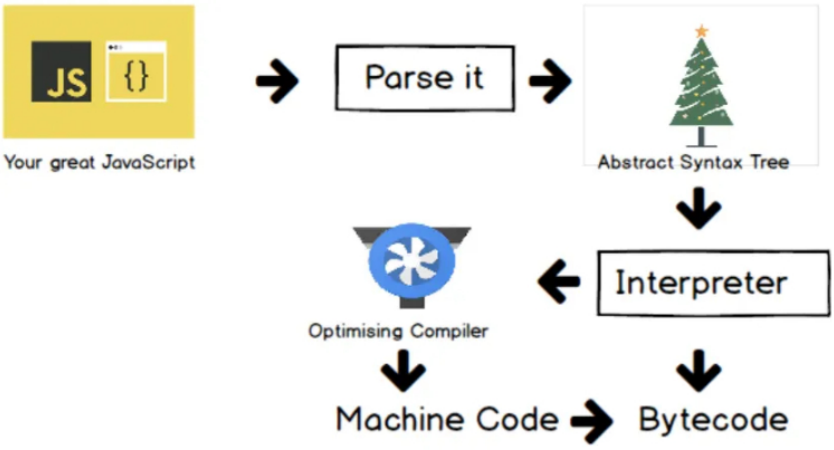
- Lightweight, interpreted language
- Object-oriented
- First-class functions
- Platform-independent

How it works





- JavaScript runs in browsers and on servers (Node.js).
- It is executed by JavaScript engines like V8 (Chrome).



Where to use

- Web development (client-side & server-side)
- Mobile apps
- Desktop apps

JavaScript V/S Java

`JavaScript`	`Java`
Interpreted	Compiled
Dynamically typed	Statically typed
Prototype-based	Class-based
Runs in browser	Runs on JVM
Single-threaded	Multi-threaded

Setup Environment

You can run JS in browsers or use Node.js for server-side.



Output Syntax

```
const id = 45465
var user = "sanjeev"
```

```
let pwd = "1234"
simple = "good"

//variable
console.log(simple)
//string interpolation
console.log('hello')
console.log('hello','world')
console.log('hi'+'user')
console.log('testing',pwd)
console.log(`Hello, ${user}!`);

console.table([id,user,pwd,simple])
```

Statements

Statements are instructions executed by the JS engine.

```
let x = 5;
```

Comments

```
// Single-line comment
/* Multi-line
    comment */
```

---

2. Variables, Data Types , Literals , Keywords and Identifiers

- support `Dynamic` and `weak(implicit type conversion)` typing

```
//dynamic example
let x = 45;
x = 'sanjeev';

//weak example
let x = 45;
y = x + '1';
console.log(y) // output:451 (number 45 is converted to string '45' and concatenated with '1')
```

Variables

```
Variables are containers for storing data values. In JavaScript, you can declare variables using `var`, `let`, or `const`.
```

`var` vs `let` vs `const`

- `var` : function-scoped, can be re-declared and re-assigned
- `let` : block-scoped, cannot be re-declared in same scope but can be re-assigned
- `const` : block-scoped, cannot be re-assigned and re-declared

```
var a = 1;
var x; //declaration with undefined value
x = 45; //initialization
let b = 2;
const c = 3; // must be initialized and declared together

// valid identifiers|variable name
let myVar = 10;
let $dollar = 20;
let _underscore = 30;

// invalid identifiers|variable name
let 1stVar = 10; // starts with digit
let my-var = 20; // contains hyphen
let for = 30; // reserved keyword
let hello world = 40; // contains space
```

Identifiers

Names for variables, functions, etc. Must start with a letter, `$`, or `_`.

Rules for Identifiers

- Can contain letters, digits, `$`, and `_`
- Cannot start with a digit
- Cannot be a reserved keyword
- Case-sensitive
- Identifiers can be used in various places

```
const decl = 1; // Variable declaration (may also be `let` or `var`)
function fn() {} // Function declaration
const obj = { key: "value" }; // Object keys
// Class declaration
class C {
```

```
  #priv = "value"; // Private field
}
lbl: console.log(1); // Label
```

Literals

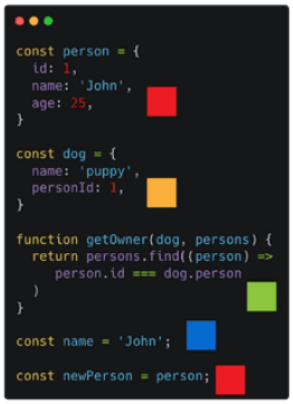
```
//number literal
45
78
```

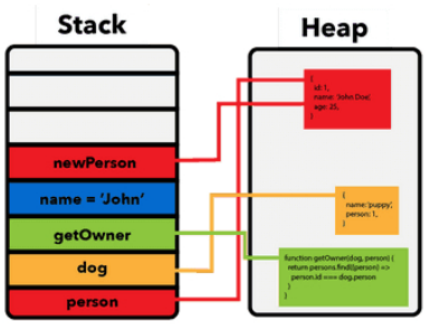
template literals (adv js)

```
`Hello, ${name}!`
```

Memory Allocation and Working

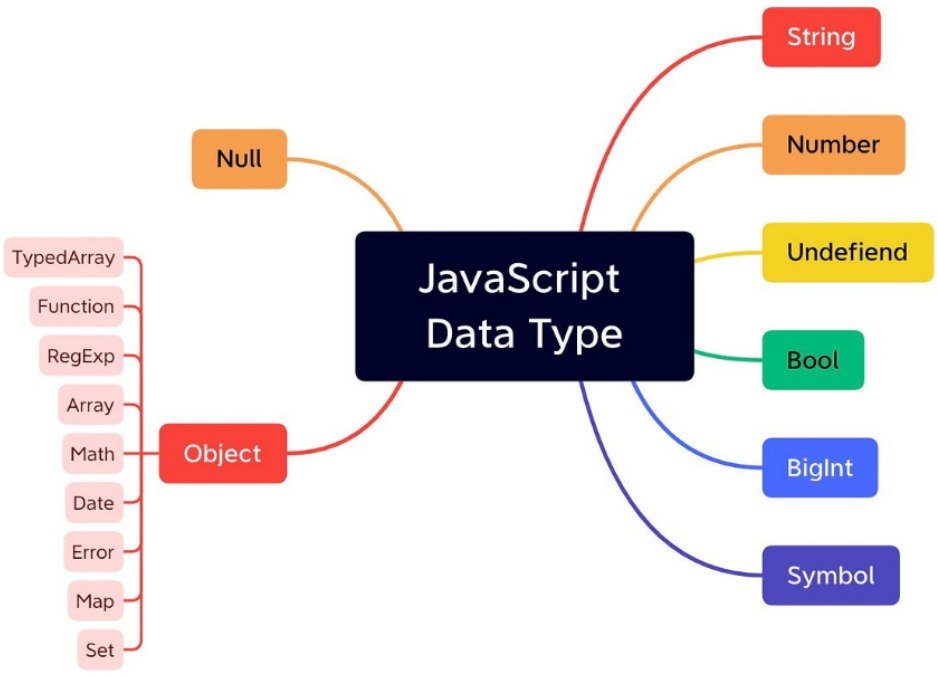
Variables are stored in memory and managed by the JS engine.





Data types

- `Primitive` : Number , String , Boolean , Null , Undefined , Symbol , BigInt
- `Non-primitive` : Object , Array , Function




```
// primitive
```

```
// 7 types : String, Number, Boolean , null , undefined , Symbol, BigInt

null  //stand alone value
undefined
symbol => unique

console.log(typeof null);  //null is object type

console.log(typeof(undefined)); // undefined is undefined type
const id = Symbol('123')
const anotherId = Symbol('123')
console.log(id === anotherId)
console.log(id == anotherId)
Symbol.keyFor(Symbol.for("tokenString")) === "tokenString"; // true

const Bigint = 546454564878954545454564654545n //if need exact same number use 'n' in last otherwise give scientific notation
console.log(Bigint)



//Reference ( Non primitive) => Array, Objects, Functions


let array = [5,5,8,45]
console.log(array , typeof array)

let object = {
    name: 'John',
    age: 25
}

console.log(object,typeof object)

 function news(params) {
    console.log('welcome')
}

console.log(news(),news,typeof news)
```

Keywords

```
Reserved words with special meaning in JS.
```

```
// Variable declaration
var, let, const

// Control flow
if, else, switch, case, default, for, while, do, break, continue

// Functions and classes
function, class, extends, super, this, new, return, yield

// Error handling
try, catch, finally, throw

// Modules
import, export

// Asynchronous operations
async, await

// Other
typeof, instanceof, delete, void, in, debugger, with
```

3. Operators

Types

Arithmetic Operators

Operator	Description	Example
`+`	Addition	`5 + 3`
`-`	Subtraction	`5 - 3`
`*`	Multiplication	`5 * 3`
`/`	Division	`5 / 3`
`%`	Modulus	`5 % 3`
`**`	Exponentiation	`5 ** 3`
`++`	Increment	`i++`
`--`	Decrement	`i--`

Assignment Operators

Operator	Description	Example
`=`	Assignment	`x = 5`
`+=`	Add and assign	`x += 3`
`-=`	Subtract and assign	`x -= 3`
`*=`	Multiply and assign	`x *= 3`
`/=`	Divide and assign	`x /= 3`
`%=`	Modulus and assign	`x %= 3`
`**=`	Exponentiate and assign	`x **= 3`
`<<=`	Left shift and assign	`x <<= 2`
`>>=`	Right shift and assign	`x >>= 2`
`>>>=`	Unsigned right shift and assign	`x >>>= 2`
`&=`	Bitwise AND and assign	`x &= 3`
`^=`	Bitwise XOR and assign	`x ^= 3`
`|=`	Bitwise OR and assign	`x |= 3`

Comparison Operators

Operator	Description	Example
`==`	Equal	`5 == '5'`
`===`	Strict equal	`5 === '5'`
`!=`	Not equal	`5 != '5'`
`!==`	Strict not equal	`5 !== '5'`
`>`	Greater than	`5 > 3`
`<`	Less than	`5 < 3`
`>=`	Greater than or equal	`5 >= 5`
`<=`	Less than or equal	`5 <= 5`

Logical Operators

Operator	Description	Example
`&&`	Logical AND	`true && false`
`||`	Logical OR	`true || false`
`!`	Logical NOT	`!true`

Bitwise Operators

Operator	Description	Example
`&`	Bitwise AND	`5 & 3` /* 0101 & 0011 ==> 0001 [1] */
`|`	Bitwise OR	`5 | 3` /* 0101
`^`	Bitwise XOR	`5 ^ 3` /* 0101 ^ 0011 ==> 0110 [6] */
`~`	Bitwise NOT	`~5` /* ~0101 ==> 1010 [-6] */
`<<`	Left shift	`5 << 1` /* 0101 << 1 ==> 1010 [10] */
`>>`	Right shift	`5 >> 1` /* 0101 >> 1 ==> 0010 [2] */
`>>>`	Unsigned right shift	`5 >>> 1` /* 0101 >>> 1 ==> 0010 [2] */

Special Operators

Operator	Description	Example
`? :`	Ternary	`x > 5 ? 'yes' : 'no'`
`,`	Comma	`let x = (1, 2, 3);`
`??`	Nullish coalescing	`x ?? 'default'`
`?.`	Optional chaining	`obj?.prop`
`in`	Property existence	`'prop' in obj`
`instanceof`	Instance check	`obj instanceof ClassName`
`typeof`	Type check	`typeof x === 'string'`
`.`	Member access	`obj.prop`
`[]`	Element access	`arr[0]`
`()`	Function call	`func()`

`new`	Object instantiation	`new ClassName()`
`this`	Current context	`this.prop`
`delete`	Property deletion	`delete obj.prop`
`...`	Spread/rest	`let arr2 = [...arr1];`
`destructuring`	Unpacking values	`let {a, b} = obj;`
`=>`	Arrow function	`const fn = (x) => x * 2;`
`yield`	Generator function	`function* gen() { yield 1; }`

```
let sum = 5 + 3; // 8
let isEqual = (5 === '5'); // false

// Operations
// null convert into number when use comparision operator excel equality
console.log(null > 0) // false
console.log(null == 0)  //false
console.log(null >= 0) //true

console.log(undefined > 0) // false
console.log(undefined == 0)  //false
console.log(undefined >= 0) //false

// === strict check with datatypes
```

Expressions vs Operands vs Operators

- `Expression` : a combination of values, variables, and operators
- `Operand` : values or variables
- `Operator` : symbols that perform operations

Conversion

```
// Type conversion examples
let x = '45';
let num = Number(x); // converts string to number
let str = String(123); // converts number to string
let bool = Boolean(1); // converts number to boolean
x = parseInt('123'); // converts string to integer
console.log(typeof num); // number
```

Operator Precedence

Determines the order in which operators are evaluated.

Precedence	Operators
Highest	`()` , `[]` , `.`
	`!` , `~` , `++` , `--` , `typeof` , `void` , `delete`
	`**`
	`*` , `/` , `%`
	`+` , `-`
	`<<` , `>>` , `>>>`
	`<` , `<=` , `>` , `>=` , `in` , `instanceof`
	`==` , `!=` , `===` , `!==`
	`&`
	`^`
	`|`
	`&&`
	`||`
	`??`
	`? :`
	`=` , `+=` , `-=` , `*=` , `/=` , `%=`
Lowest	`,`
	`=>`

4. Control Flow Statements

```
Control flow statements allow you to control the execution of code based on conditions.
```

if, if-else, if-else-if ladder

```
 // falsy value
// false , 0 , -0 , BigInt 0n , "",null,undefined,NaN

//truthy vlaues
// '0','false'," ",[],{},function(){}


//single line statements

if(false) console.log('hi'); else console.log('my')

false?console.log('1'):console.log('45')//  ternary operator

// Nullish Coalescing Operator (??) : null undefined
console.log(null ?? 'default value1');
console.log(undefined ?? 'default value');
console.log(45 ?? ' 41');
console.log(null ?? 45 ?? 10);


// multiple line without {} use ,
if(true) console.log('hi'),
 console.log('my')

if (x > 0) {
  console.log("Positive");
} else if (x < 0) {
  console.log("Negative");
} else {
  console.log("Zero");
}
```

switch-case

```
switch(day) {
  case "Mon":
     console.log("Monday");
     break;
  default:
     console.log("Other day");
}
```

Escape Sequences

Escape sequence	use
\0	null character (U+0000 NULL)
\'	single quote (U+0027 APOSTROPHE)
\"	double quote (U+0022 QUOTATION MARK)
\\	backslash (U+005C REVERSE SOLIDUS)
\n	newline (U+000A LINE FEED; LF)
\r	carriage return (U+000D CARRIAGE RETURN; CR)
\v	vertical tab (U+000B LINE TABULATION)
\t	tab (U+0009 CHARACTER TABULATION)
\b	backspace (U+0008 BACKSPACE)
\f	form feed (U+000C FORM FEED)
\ followed by a line terminator	empty string

```
const longString =
  "This is a very long \n string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable.";
```

5. Loops

```
Loops are used to execute a block of code repeatedly.Loops repeat code while a condition is true.
```

Initialization, Condition, Increment/Decrement

- `Initialization` : set starting point
- `Condition` : loop continues while true
- `Increment/Decrement` : update loop variable

for

```
for(let i = 0; i < 5; i++) {
  console.log(i);
}
```

while

```
let i = 0;
while(i < 5) {
  console.log(i);
  i++;
}
```

do-while

```
let i = 0;
do {
  console.log(i);
  i++;
} while(i < 5);
```

for-in

Iterates over object properties.

```
let obj = {a:1, b:2};
for(let key in obj) {
  console.log(key, obj[key]);
}
```

for-of

Iterates over iterable objects (arrays, strings).

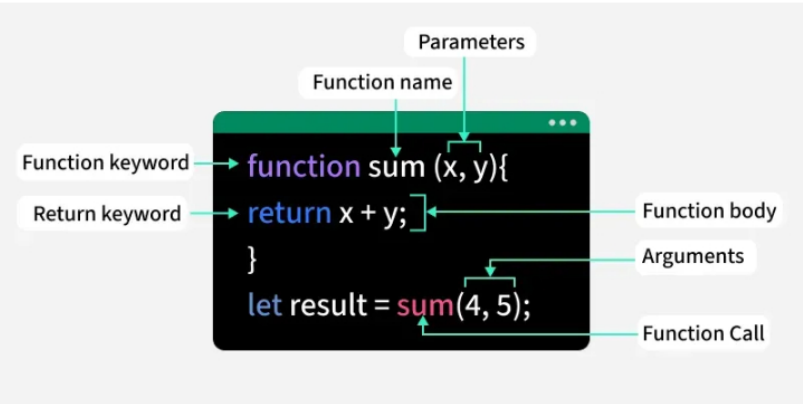
```
let arr = [1,2,3];
for(let val of arr) {
  console.log(val);
}
```

break vs continue

- `break` : exits loop
- `continue` : skips to next iteration

6. Functions

```
Functions are reusable blocks of code that perform a specific task.
```



Important Terms

Parameters vs Arguments

- Parameters: variables in function definition
- Arguments: values passed to function

Return statement

> Returns a value from a function.

Function call

> Invokes a function to execute its code.

Types of Functions

Built-in Functions

```
// Example of a built-in function
let result = Math.max(5, 10);
```

User-defined Functions

```
function greet(name) {
  return "Hello, " + name;
}
console.log(greet("Alice"));
```

Ways to define a function

1. Take something,return something

```
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // 5
```

2. Take something, return nothing

```
function greet(name) {
  console.log("Hello, " + name);
}
greet("Alice"); // Hello, Alice
```

3. Take nothing, return something

```
function getCurrentYear() {
  return new Date().getFullYear();
}
console.log(getCurrentYear()); // 2024
```

4. Take nothing, return nothing

```
function sayHello() {
  console.log("Hello, World!");
}
sayHello(); // Hello, World!
```

Function Declarations

Normal function

```
function add(a, b) {
  return a + b;
}
```

Recursion

Function calling itself.

```
function factorial(n) {
  if(n <= 1) return 1;
  return n * factorial(n - 1);
}
```

Function Expression | Anonymous Function

Function assigned to a variable without a name.

```
const add = function(a, b) {
  return a + b;
};
```

Arrow Functions

Shorter syntax for function expressions.

> `Note` : Arrow functions do not have their own `this` context.

```
const add = (a, b) => a + b;
```

Function `Scope` and `Closures`

Functions create their own scope. Closures allow functions to access variables from outer scopes.

```
function outer() {
  let x = 10;
  function inner() {
     console.log(x);
  }
  return inner;
}
const fn = outer();
fn(); // 10
```

`Callback` Functions

Functions passed as arguments to other functions.

```
function greet(name, callback) {
  callback(name);
}
greet("Alice", (n) => console.log("Hello " + n));
```

`IIFE` (Immediately Invoked Function Expression)

```
(function() {
  console.log("IIFE");
})();
```

7. Arrays

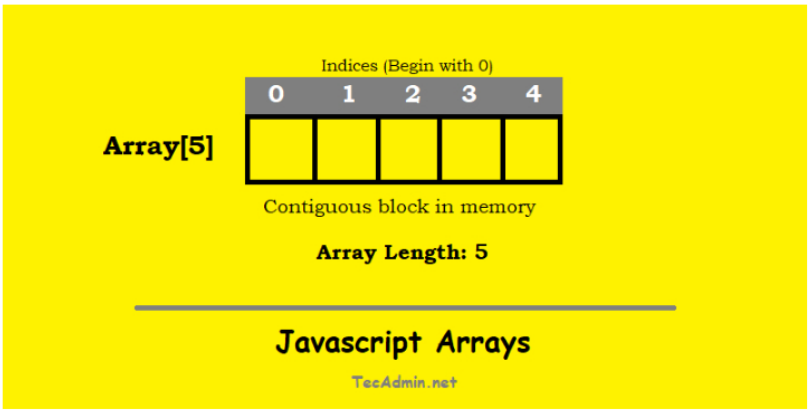
```
Arrays are used to store multiple values in a single variable. It store data in contiguous memory locations.
```

Creating Arrays

```
// using [] brackets
let emptyArr = [];// empty array
let arr = [1, 2, 3]; // with elements

//or using Array constructor
let arr2 = new Array(1, 2, 3);
let arr3 = new Array(5); // creates an array of length 5 with undefined values

// multi-dimensional array
let matrix = [[1,2],[3,4]];// 2D array
let matrix3D = [[[1]], [[2]]];// 3D array
let matrix2D = new Array([[1,2],[3,4]]);// 2D array
```



Accessing Elements

```
//syntax:- zero-based index
// for 1D array: arr[index]
// for multi-dimensional array: arr[row][col]
console.log(arr[0]); // 1D
console.log(matrix[0][1]); // 2D
```

Manipulating Arrays

```
arr[0] = 10; // update
console.log(arr[0]); // 10

arr2D[0][1] = 20; // update 2D
```

Destructuring Arrays

```
const fruits = ["apple", "banana", "cherry"];

// Basic destructuring
const [firstFruit, secondFruit] = fruits;
console.log(firstFruit);  // "apple"
console.log(secondFruit); // "banana"

// Skipping values
const [,, thirdFruit] = fruits;
console.log(thirdFruit);  // "cherry"

// Rest operator to gather remaining elements
const [a, ...restOfFruits] = fruits;
console.log(a);             // "apple"
console.log(restOfFruits);  // ["banana", "cherry"]

// Swapping values
let x = 1;
let y = 2;
[x, y] = [y, x];
console.log(x); // 2
console.log(y); // 1
```

Array Methods

Method	Description	Example
`push()`	Adds element to end	arr.push(4);
`pop()`	Removes last element	arr.pop();
`shift()`	Removes first element	arr.shift();
`unshift()`	Adds element to start	arr.unshift(0);
`indexOf()`	Finds index of element	arr.indexOf(2);
`slice()`	Extracts part of array	arr.slice(1,3);
`splice()`	Adds/removes elements	arr.splice(1,1);
`forEach()`	Iterates over elements	arr.forEach(x => console.log(x));
`map()`	Transforms elements	arr.map(x => x * 2);
`filter()`	Filters elements	arr.filter(x => x > 1);
`reduce()`	Reduces to single value	arr.reduce((a,b) => a + b);
`sort()`	Sorts elements	arr.sort();
`reverse()`	Reverses array	arr.reverse();
`join()`	Joins elements into string	arr.join(", ");
`concat()`	Merges arrays	arr.concat([4,5]);
`includes()`	Checks for element	arr.includes(2);
`find()`	Finds first matching element	arr.find(x => x > 1);
`findIndex()`	Finds index of first matching element	arr.findIndex(x => x > 1);

8. Strings

```
Strings are used to store and manipulate text. Strings are immutable, meaning they cannot be changed after creation.
```

Creating Strings

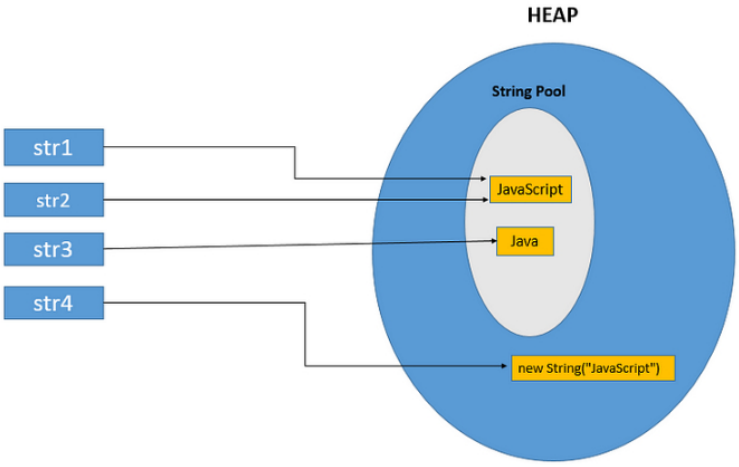
```
//simple way
let str = "Hello";

//or using String constructor
let str2 = new String("Hello");
```

String Manipulation

```
str[0] = "h"; // does not change
console.log(str[0]); // "H"
str2[0] = "h"; // does not change
```

```
console.log(str2[0]); // "H"
```



String Methods

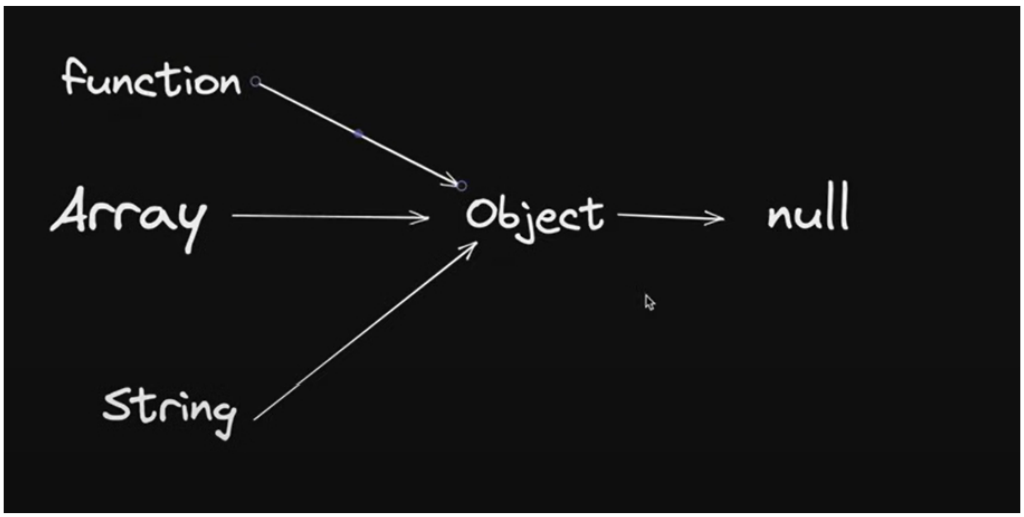
- `length`, `toUpperCase()`, `toLowerCase()`, `slice()`, `replace()`, `split()`, `concat()`

```
console.log(str.toUpperCase()); // "HELLO"
```

`Template Literals`

```
let name = "Alice";
console.log(`Hello, ${name}!`);
```

9. Objects



Creating Objects

```
let person = {name: "Bob", age: 25};
```

Accessing Properties

```
console.log(person.name); // "Bob"
// or
console.log(person["age"]); // 25
```

Object Methods

```
person.greet = function() {
  console.log("Hello!");
};

person.greet();
```

Destructuring Objects

```
const user = {
  name: "Alice",
  age: 30,
  city: "New York"
};

// Basic destructuring
const { name, age } = user;
console.log(name); // "Alice"
console.log(age);  // 30

// Renaming variables
const { name: userName, city: userCity } = user;
console.log(userName); // "Alice"
console.log(userCity); // "New York"

// Default values
const { country = "USA" } = user; // 'country' is not in 'user'
console.log(country); // "USA"

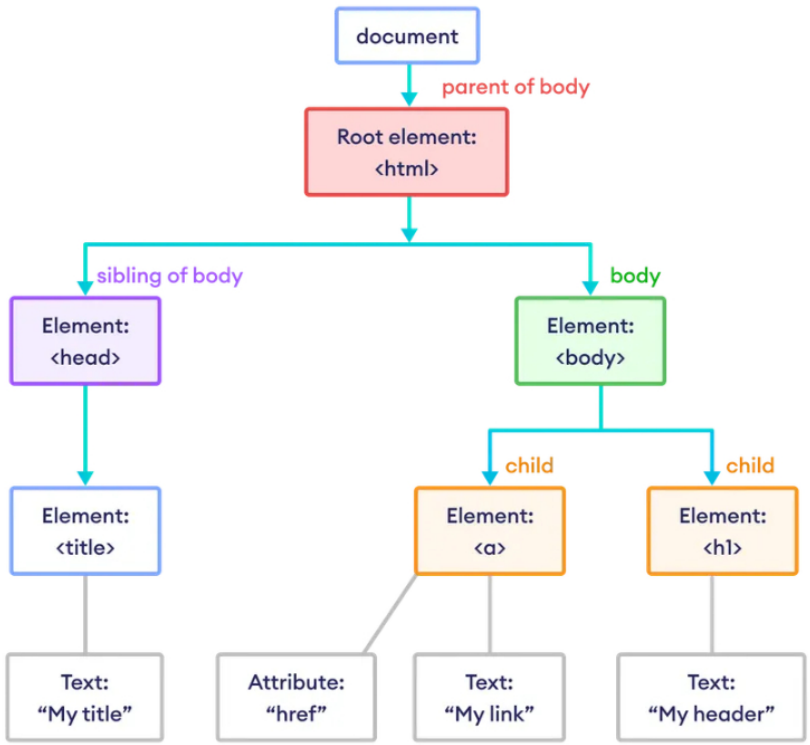
// Destructuring nested objects
const profile = {
  id: 123,
  details: {
    email: "alice@example.com",
    phone: "123-456-7890"
  }
};
const { details: { email } } = profile;
console.log(email); // "alice@example.com"

//paranthesis requires
({ a, b } = { a: 1, b: 2 });
console.log(a,b)
```

10. DOM Manipulation

```
The Document Object Model (DOM) is a programming interface for web documents.

It represents the page so that programs can change the document structure, style, and content.
```



Selecting Elements Methods

- `querySelector()` returns the first matching element or null.
- `querySelectorAll()` returns all matching elements as a NodeList.
- `getElementById()` returns a single element by its ID.

- `getElementsByClassName()` returns a live HTMLCollection of elements with the specified class
- `getElementsByTagName()` returns a live HTMLCollection of elements with the specified tag name.

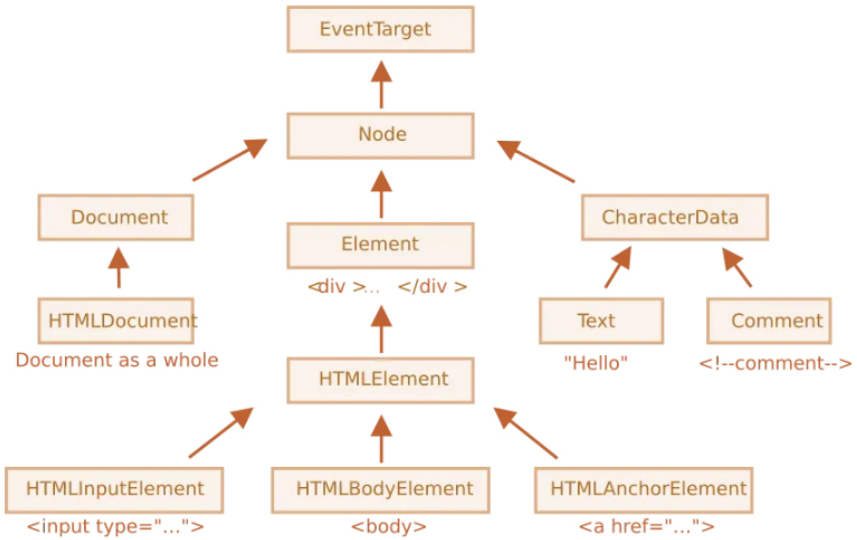
Single Element

```
let el = document.getElementById("myId");// HTMLElement
el = document.querySelector(".myClass");// HTMLElement
```

Multiple Elements

```
let els = document.getElementsByClassName("myClass"); // HTMLCollection
els = document.getElementsByTagName("div"); // HTMLCollection
els = document.querySelectorAll(".myClass"); // NodeList
```

- **HTMLCollection:** live collection, updates with DOM changes
- **NodeList:** It is both static & live collection, may does not update with DOM changes

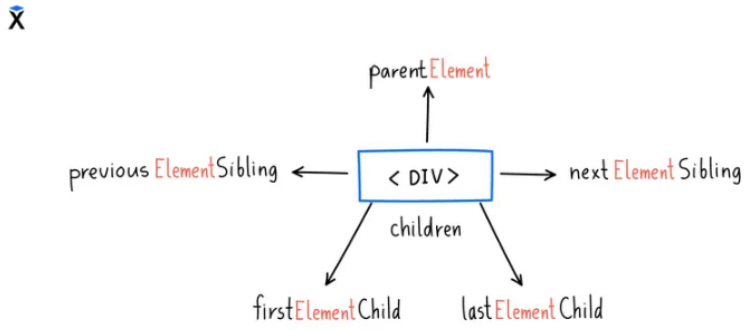


Types of Nodes

Node Type	Description	Example
Element Node	Represents an HTML element	`<div>`
Text Node	Represents text content	`Hello`
Comment Node	Represents a comment	`<!-- comment -->`
Document Node	Represents the entire document	`document`
Child Node	Represents a child of an element	`<p>` inside `<div>`
Parent Node	Represents the parent of an element	`<div>` is parent of `<p>`
Sibling Node	Represents a node at the same level	`<h1>` and `<p>` inside `<div>`
Attribute Node	Represents an attribute of an element	`class="myClass"`

Traversing the DOM

`Definition:` Navigating through the elements and nodes in the DOM tree.



Property & Methods	Description	Example
`parentNode`	Gets the parent node of an element	`let parent = el.parentNode;`
`childNodes`	Gets all child nodes of an element	`let children = el.childNodes;`
`firstChild`	Gets the first child node of an element	`let firstChild = el.firstChild;`
`lastChild`	Gets the last child node of an element	`let lastChild = el.lastChild;`
`nextSibling`	Gets the next sibling node of an element	`let nextSibling = el.nextSibling;`
`previousSibling`	Gets the previous sibling node of an element	`let previousSibling = el.previousSibling;`
`children`	Gets all child elements of an element	`let allChildren = el.children;`
`firstElementChild`	Gets the first child element of an element	`let firstElem = el.firstElementChild;`
`lastElementChild`	Gets the last child element of an element	`let lastElem = el.lastElementChild;`
`nextElementSibling`	Gets the next sibling element of an element	`let nextElem = el.nextElementSibling;`
`previousElementSibling`	Gets the previous sibling element of an element	`let prevElem = el.previousElementSibling;`
`hasChildNodes()`	Checks if an element has child nodes	`let hasChildren = el.hasChildNodes();`

```
let parent = el.parentNode;
let children = el.childNodes; // NodeList of child nodes
let firstChild = el.firstChild;
let lastChild = el.lastChild;
let nextSibling = el.nextSibling;
let previousSibling = el.previousSibling;
let allChildren = el.children; // HTMLCollection of child elements
```

Changing Content and Styles

Property & Methods	Description	Example
`textContent`	Sets or gets the text content of an element	`el.textContent = "New Text";`
`innerHTML`	Sets or gets the HTML content of an element	`el.innerHTML = "<b>Bold</b>";`
`innerText`	Sets or gets the visible text content of an element	`el.innerText = "Visible Text";`
`style`	Sets or gets the inline styles of an element	`el.style.color = "red";`

```
el.textContent = "New Text";
el.style.color = "red";
```

Modifying Elements

Property & Methods	Description	Example
`className`	Sets or gets the class attribute of an element	`el.className = "active";`
`classList`	Adds, removes, or toggles classes on an element	`el.classList.add("active");`
`setAttribute()`	Sets an attribute on an element	`el.setAttribute("data-id", "123");`
`getAttribute()`	Gets an attribute from an element	`let id = el.getAttribute("data-id");`
`removeAttribute()`	Removes an attribute from an element	`el.removeAttribute("data-id");`
`appendChild()`	Adds a child element to an element	`el.appendChild(newEl);`
`removeChild()`	Removes a child element from an element	`el.removeChild(childEl);`
`replaceChild()`	Replaces a child element with another	`el.replaceChild(newEl, oldEl);`
`insertBefore()`	Inserts a new element before a specified child	`el.insertBefore(newEl, referenceEl);`
`createElement()`	Creates a new element	`let div = document.createElement("div");`
`createTextNode()`	Creates a new text node	`let text = document.createTextNode("Hello");`
`createComment()`	Creates a new comment node	`let comment = document.createComment("This is a comment");`
`remove()`	Removes the element from the DOM	`el.remove();`
`removeParentNode()`	Removes the parent element from the DOM	`el.parentNode.remove();`

```
let div = document.createElement("div");
document.body.appendChild(div);
document.body.removeChild(div);
```

Events

```
Events are actions or occurrences that happen in the system you are programming, which the system tells you about so your code
```

Event Types

Event Type	Description	Example
`click`	Triggered when an element is clicked	`el.addEventListener("click", fn);`
`dblclick`	Triggered when an element is double-clicked	`el.addEventListener("dblclick", fn);`
`mouseover`	Triggered when the mouse pointer is over an element	`el.addEventListener("mouseover", fn);`
`mouseout`	Triggered when the mouse pointer leaves an element	`el.addEventListener("mouseout", fn);`
`keydown`	Triggered when a key is pressed down	`el.addEventListener("keydown", fn);`

```
<!-- html file -->
<button id="myBtn">Click Me</button>
<button id="myBtn2" onclick="change()">Click Me</button>
```

Event `Handlers`

```
// js file
let el = document.getElementById("myBtn");
el.onclick = function() {
  alert("Clicked!");
};

//or
let change = function() {
  alert("Clicked!");
};
```

Event `Listeners`

```
let el = document.getElementById("myBtn");
el.addEventListener("click", function() {
  alert("Clicked!");
});
```

Event `Object`

Property & Methods	Description	Example
`type`	Gets the type of the event	`let type = event.type;`
`target`	Gets the target element of the event	`let target = event.target;`
`currentTarget`	Gets the current element handling the event	`let current = event.currentTarget;`
`preventDefault()`	Prevents the default action of the event	`event.preventDefault();`
`stopPropagation()`	Stops the event from bubbling up the DOM	`event.stopPropagation();`
`removeEventListener()`	Removes an event listener from an element	`el.removeEventListener("click", fn);`
`addEventListener()`	Adds an event listener to an element	`el.addEventListener("click", fn);`
`createEvent()`	Creates a new event	`let evt = document.createEvent("Event");`
`dispatchEvent()`	Dispatches an event to an element	`el.dispatchEvent(evt);`
`bubbles`	Indicates if the event bubbles up through the DOM	`let doesBubble = event.bubbles;`
`cancelable`	Indicates if the event can be canceled	`let isCancelable = event.cancelable;`
`initEvent()`	Initializes a newly created event	`evt.initEvent("click", true, true);`

```
let evt = document.createEvent("Event");
// Initialize the event --> event type, bubbling, cancelable
evt.initEvent("click", true, true);
el.dispatchEvent(evt);
```

Event `Propagation`

```
Event propagation is the order in which events are received on the page. There are two main phases:
```

- `Capturing Phase` : The event starts from the window and goes down to the target element.
- `Bubbling Phase` : The event starts from the target element and bubbles up to the window.

```
document.body.addEventListener("click", function(event) {
  console.log("Body clicked");
});

document.getElementById("myBtn").addEventListener("click", function(event) {
  console.log("Button clicked");
});
```

Event `Delegation`

```
Event delegation is a technique of using a single event listener to manage all events of a particular type on multiple child el
```

```
document.getElementById("parent").addEventListener("click", function(event) {
  if (event.target.matches(".child")) {
    console.log("Child clicked");
  }
});
```